

Embedded Probe Sampling for Measuring Convergence in Online Deliberative Events

Method report and implementation specification

- **Author:** Tal Yaron, with method review by Claude (Anthropic)
- **Date:** 2026-07-24
- **Status:** Design approved, not yet implemented
- **Applies to:** Freedi Mass-Consensus app (`apps/mass-consensus`)

Abstract

Deliberative events aim to move a group toward broadly supported proposals, yet most platforms cannot answer the basic outcome question: *did support for the leading proposals actually grow during the event?* Pre/post surveys answer it at the cost of extra participant burden and low response rates. We propose an **embedded probe design**: during the normal evaluation flow, in which each participant repeatedly receives small batches of proposals to rate, one slot per batch is reserved for a **probe** — a proposal drawn from the current top set ranked by average evaluation (avg-eval). The statistic of interest is the **per-user, within-batch delta**: the participant's rating of the probe minus their mean rating of the sampled items in the same batch. Tracked across successive batches, a rising delta indicates that the leading edge of the proposal pool is pulling away from the field — a live convergence signal — at zero additional participant burden. We describe the design, its identifying assumptions, known threats to validity and their mitigations, a power analysis for a typical event ($N \approx 100$), and a concrete implementation plan for the Freedi Mass-Consensus codebase.

1. Background and motivation

1.1 The measurement gap

In Freedi Mass-Consensus (MC) events, participants submit proposals and repeatedly evaluate small samples of other participants' proposals on a $-1...+1$ scale. The platform aggregates these into per-statement statistics — average evaluation, and the consensus index $C_p = \mu_p - t \cdot SEM^*$ (mean penalized by uncertainty; see `packages/shared-types/src/utils/consensusCalculation.ts`).

These aggregates rank proposals well, but they do not measure whether the *group's state of agreement* improved between the start and the end of an event. Two problems block the naive approaches:

1. **A separate pre/post questionnaire** adds burden, suffers attrition, and measures a different task (abstract attitude items) than the one participants actually perform (rating concrete proposals).
2. **Reading trends off the organic rating stream** is underpowered: under (smart) random sampling each individual proposal receives only a few ratings per time-slice, so per-item trajectories are noise. Worse, the adaptive sampler deliberately *stops serving* proposals once their estimate is stable (`isStable` early-

stopping in `apps/mass-consensus/src/lib/utils/sampling.ts:346`), so precisely the proposals we care most about — the established leaders — stop being measured at all.

1.2 The idea

Participants already rate batches of 6 proposals, several times per event. We repurpose **one slot in six** as a measurement instrument:

In every batch after the first, 5 slots are filled by the normal sampler and 1 slot is filled by a proposal from the **current top set** (highest avg-eval among eligible proposals). The participant cannot distinguish the probe from the other items.

Because every participant rates both probe and non-probe items *in the same batch*, each batch yields a self-contained, within-person comparison. The probes do not create the measurement — time-0 ratings of eventually-top items exist in the organic stream — they create the **statistical power** for it, and they keep the leaders under continuous measurement after the adaptive sampler retires them.

A deliberate design choice: the probe ratings are **not** excluded from the live aggregates. A top proposal that stops earning support is therefore demoted by the probe ratings themselves — the oversampling doubles as a stress test of the leaders, making the top set self-correcting.

2. Method

2.1 Definitions

- **Batch** — one serving of 6 proposals to one participant (existing MC unit; `batchCount` in `SolutionFeedClient.tsx`). Batch index r plays the role of "round."
- **avg-eval** — `statement.evaluation.averageEvaluation` = $\Sigma E / n$, the plain mean of ratings. Chosen over `C_p` as the probe-selection metric so that leaders which stop earning support fall out of the top set directly, without the confidence penalty masking the drop.
- **Top set $T(t)$** — at serving time t , the k eligible proposals with the highest avg-eval (recommended $k = 3-5$). The top set is **dynamic**: it is re-derived at every serving, and yesterday's leader may drop out.
- **Eligibility floor** — a proposal enters $T(t)$ only if `numberOfEvaluators` $\geq$ `n_min` (recommended `n_min` = 10–15).
- **Probe slot** — 1 of the 6 slots in batches $r \geq 2$, filled by rotation over $T(t)$, excluding the participant's own proposals and any proposal they have already evaluated.

2.2 The statistic

For participant u in batch r , let $p(u, r)$ be their rating of the probe and $S(u, r)$ the mean of their ratings of the 5 sampled items in the same batch. The per-user delta is:

$$\delta(u, r) = p(u, r) - S(u, r)$$

The event-level convergence curve is the mean across participants active in that batch index:

$$\Delta(r) = \text{mean over } u \text{ of } \delta(u, r), \quad r = 2, 3, 4, \dots$$

with a confidence interval from the between-user standard error. For inferential use, a mixed model is the clean formulation:

$$\text{rating} \sim \text{slotType} \times \text{batchIndex} + (1 \mid \text{userId}) + (1 \mid \text{statementId})$$

where $\text{slotType} \in \{\text{probe}, \text{sampled}\}$; the $\text{slotType} \times \text{batchIndex}$ interaction is $\Delta(r)$'s trend.

2.3 What the curve means — index semantics

Because the top set rotates, $\Delta(r)$ tracks a **position, not particular items** — like a stock index that stays meaningful while its constituents change. $\Delta(r)$ answers: "*how far ahead of the field is whoever is leading right now?*" A rising $\Delta(r)$ means the leading edge is pulling away; it must **not** be read as "those specific proposals gained support."

Note that $\Delta(r) > 0$ in any single batch is expected by construction — items selected for high observed avg-eval will outscore a random draw even in a static world. **The level of Δ carries no information; only its trajectory does**, and even the trajectory has mechanical components that must be controlled (§3).

2.4 Companion metric: top-set churn

Report alongside $\Delta(r)$ the **churn**:

$$\text{churn}(r) = 1 - |T(r) \cap T(r-1)| / |T(r)|$$

the fraction of the top set replaced since the previous batch epoch. Churn disambiguates the headline result:

Pattern	Reading
Δ rising, churn falling	Convergence. The group agrees more, on a settling set — the strong result.
Δ rising, churn high	Healthy exploration: the sampler keeps surfacing new winners; no settlement yet.
Δ flat/falling, churn low	Stagnation: a stable top set that is not gaining ground.
Δ falling, churn high	Contested field / polarization; inspect dispersion.

2.5 Secondary outcome: dispersion

Convergence is as much a tightening of the distribution as a rise in the mean. Since `sumSquaredEvaluations` and `standardDeviation` are already maintained per statement, additionally report the standard deviation of probe ratings per batch index. A rising Δ with rising probe-rating dispersion signals a hardening minority — the pattern deliberation most needs to detect.

3. Threats to validity and mitigations

3.1 Regression to the mean (selection on the measured variable)

Selecting by observed avg-eval and then measuring avg-eval guarantees that items selected on few, noisy ratings will score lower on re-measurement — pure sampling arithmetic, no opinion change required. Because estimate precision grows over the event, this artifact is strongest early and fades later, painting a **spurious upward drift** on $\Delta(r)$.

Mitigation: the eligibility floor ($n_{\min} \geq 10\text{--}15$ evaluations). Selection then operates on estimates whose SEM is small relative to the between-item spread, removing most of the artifact. Report $n_{\min}$ with any published curve.

3.2 Baseline pool drift

The random-slot baseline $S(u,r)$ is drawn from a pool that fills with fresh, mostly unrated proposals as the event runs; its mean can sink over batches, inflating Δ for free.

Mitigation: the within-user, within-batch construction of $\delta(u,r)$ uses each participant's *own* same-batch sampled mean as the baseline, so between-batch pool composition cancels at the individual level. Residual drift affecting probes and samples differently is absorbed by the item random effect in the mixed model.

3.3 Attrition and population change

Batch-4 raters are the engaged survivors, not the batch-1 population. Comparisons of raw means across batches are contaminated.

Mitigation: same as 3.2 — δ is computed within-person before aggregating. For robustness, additionally report $\Delta(r)$ restricted to the completer cohort (participants present in all batch indices analyzed).

3.4 Feedback into the live ranking

Probe ratings feed the same aggregates that define the top set. This is accepted **by design** (self-correction, §1.2), but it means probe items accumulate evaluations $\sim k\times$ faster than the field, tightening their C_p penalty term relative to others.

Mitigation: tag every evaluation record with its slot type (§5.2). The analysis can then distinguish organic from probe-driven ratings, and any future variant that excludes probes from live aggregates needs only the tag, not a schema change.

3.5 Exposure effects (limitation, not mitigated)

Probes give leaders extra exposure; mere-exposure and social-proof effects could raise ratings independently of deliberative merit. Since the MC evaluation UI shows no aggregate scores during rating (per project policy: no agreement indicators during evaluation), social proof is largely blocked; mere exposure remains a bounded, acknowledged limitation. A future randomized-holdout variant (probing for only half the participants) could quantify it.

3.6 What this design does not identify

$\Delta(r)$ is a **monitoring metric**, not a causal estimate of the deliberation's effect on opinions. The causal question ("did deliberation change minds about these proposals?") requires a frozen item cohort tracked over time. The tagging scheme (§5.2) preserves everything needed to reconstruct fixed-cohort, per-item trajectories post hoc from the same data — the option is kept open at zero design cost.

4. Power analysis

Assume a typical event: $N = 100$ participants, 4 batches each, batch size 6, probe slot in batches 2–4, top-set size $k = 4$, rating scale $-1 \dots +1$ with between-rater $SD \approx 0.5$ (empirically typical for MC data).

- Probe ratings per batch index: ~ 100 (one per active participant) $\rightarrow \sim 25$ per top-set item under rotation. Comfortably above the $n \approx 15\text{--}20$ needed for a stable item mean at this SD.
- SE of $\Delta(r)$: the paired construction has $SD(\delta) \approx 0.55\text{--}0.7$ (probe SD plus baseline-mean SD, partially correlated within rater). With 100 users, $SE(\Delta) \approx 0.06\text{--}0.07$ per batch index.
- Detectable trend: a linear change in Δ of ≈ 0.15 across batches 2 $\rightarrow$ 4 is detectable at 80% power, $\alpha = 0.05$ (two-sided) — well within the effect sizes deliberation studies report for convergence on leading options.
- Budget cost: 1 slot in 6 from batch 2 onward diverts $\approx 12.5\%$ of total event evaluations from discovery (0 of 6 in batch 1, 1 of 6 thereafter) — deliberately kept low to protect discovery of late-arriving proposals.

With $N < 40$ participants, per-item probe estimates become fragile; shrink k to 2–3 or pool batch indices in analysis.

5. Implementation in the Freedi Mass-Consensus codebase

The MC serving path today: `POST apps/mass-consensus/app/api/statements/[id]/batch/route.ts` $\rightarrow$ `getAdaptiveBatch` / `getRandomOptions` in `apps/mass-consensus/src/lib/firebase/queries.ts` $\rightarrow$ `ProposalSampler.selectForUser` (`src/lib/utils/proposalSampler.ts`) with Thompson/UCB scoring (`src/lib/utils/sampling.ts`) and cluster-diverse assembly (`src/lib/utils/diverseBatch.ts`). Evaluations are written by `POST apps/mass-consensus/app/api/evaluations/[id]/route.ts` into `Collections.evaluations` (doc ID `${userId}--${statementId}`), and aggregated server-side by the `newEvaluation` trigger $\rightarrow$ `functions/src/evaluation/statementEvaluationUpdater.ts`.

The type system already anticipates this feature: `StatementEvaluationSettings.anchored` (`packages/shared-types/src/models/evaluation/Evaluation.ts:95`) describes admin-anchored options mixed into random batches but has **no consuming implementation** in MC. The probe design is the data-driven sibling of that concept and should live beside it.

5.1 Settings (shared-types)

Add `StatementEvaluationSettingsSchema` in `packages/shared-types/src/models/evaluation/Evaluation.ts` (mirror in `delib-npm` per cross-app policy):

```
probeSampling: optional(object({
  enabled: boolean(), // default false – opt-in per question
  probeSlotsPerBatch: optional(number()), // default 1
  topSetSize: optional(number()), // k, default 4
  minEvaluationsForEligibility: optional(number()), // n_min, default 12
})),
```

5.2 Evaluation record tagging (shared-types)

Add two optional fields to `EvaluationSchema` (`Evaluation.ts:13`):

```
samplingSlot: optional(picklist(['probe', 'sampled'])), // absent = legacy/organic
batchIndex: optional(number()), // 1-based, from client batchCount
```

Both are analysis-critical and cost nothing at write time. `batchIndex` is passed through from the client (`batchCount` state in `SolutionFeedClient.tsx:416`) via the batch response → evaluation POST body.

5.3 Probe selection (server, batch route)

In `queries.ts`, add `getProbeStatement(questionId, userId, config)`:

1. Query options for the question with `evaluation.numberOfEvaluators ≥ n_min`, order by `evaluation.averageEvaluation` desc, limit ~2k. (Requires a composite index on `parentId + statementType + evaluation.averageEvaluation`; per project policy, add it surgically via `gcloud firestore indexes composite create, never --force` deploy.)
2. Filter out: the user's own statements (`creator.uid === userId`), statements the user already evaluated (the already-fetched `evaluatedIds` set — reuse it, don't re-query), cluster docs (`isCluster`), and hidden/derived docs (reuse `isServableOriginal`).
3. Take the top k as `T(t)`; pick the probe by rotation — simplest robust scheme: the eligible member of `T(t)` the user hasn't rated with the fewest probe-tagged evaluations (ties broken randomly). This spreads ~100 probes/batch-index across k items without server-side state.
4. Edge cases: if no eligible probe exists (early event, or user has rated the whole top set), return none — the batch falls back to 6 sampled slots and that user contributes no δ this batch. Never block the batch on probe availability.

In the batch route: when `probeSampling.enabled` and `batchIndex ≥ 2`, request `size - 1` items from the existing sampler (passing the probe's ID in `excludeIds` to prevent duplication), insert the probe at a **random position** in the returned array (no ordering cue), and mark it in the response payload (`slotType` per item) so the client can echo the tag into the evaluation POST. The participant-facing UI renders probe and sampled items identically.

5.4 Aggregation (functions)

No change. Probe evaluations flow into `statementEvaluationUpdater.ts` like any other — this is the self-correction mechanism. The idempotency guard, increments, and `C_p` computation are untouched. (Clusters are already skipped there; probes never target cluster docs.)

5.5 Top-set churn logging

Compute churn at serving time and persist a lightweight snapshot so the curve is reconstructable: on each batch-index epoch (or simply on every probe selection, throttled), write `{questionId, timestamp, topSetIds}` to a `probeTopSetSnapshots` subcollection under the question statement. Churn and $T(t)$ history then come from snapshots, not from re-deriving historical avg-evals (which is impossible from current aggregates alone).

5.6 Analysis

A small script (or an admin-report section, following the existing Sign document-report pattern) reads `Collections.evaluations` filtered by `parentId == questionId` and `samplingSlot != null`, groups by `evaluatorId × batchIndex`, computes $\delta(u,r)$, and outputs:

- $\Delta(r)$ with 95% CI per batch index (between-user SE);
- `churn(r)` from `probeTopSetSnapshots`;
- probe-rating SD per batch index (dispersion, §2.5);
- completer-cohort $\Delta(r)$ as a robustness column (§3.3).

The mixed model (§2.2) is the publication-grade estimate; the per-user delta table is the live dashboard number.

5.7 Rollout

1. shared-types: schema fields (§5.1, §5.2) → build package, publish to delib-npm.
2. MC server: `getProbeStatement` + batch-route integration behind `probeSampling.enabled` (default **off**, consistent with the platform's opt-in convention).
3. MC client: pass `batchIndex` + per-item `slotType` through to the evaluation POST. No UI change.
4. Firestore: add the composite index (surgically, §5.3).
5. Pilot on one internal event; verify $\Delta(1) \approx \text{undefined}$, $\Delta(2..4)$ populated, churn snapshots written, discovery coverage (evaluations per new proposal) within ~12% of a control question.
6. Analysis script + admin report.

Tests (per repo policy): unit tests for `getProbeStatement` (eligibility floor, own-statement/already-rated exclusion, rotation fairness, empty-top-set fallback) and for the δ/Δ computation, in `__tests__` beside the modules.

6. Summary

Design element	Choice	Rationale
Instrument	1 probe slot in 6, batches ≥ 2	Power for the leaders; $\approx 12.5\%$ budget cost protects discovery
Selection metric	avg-eval (not C_p)	Leaders that stop earning support fall out directly
Top set	Dynamic, re-derived each serving	The question is about the <i>position</i> (index semantics), not fixed items
Eligibility	$\geq n_{\text{min}}$ evaluations	Kills the regression-to-the-mean artifact
Statistic	$\delta(u,r)$ within-user, within-batch; $\Delta(r)$ across users	Cancels pool drift and attrition
Companion metrics	Top-set churn; probe-rating dispersion	Disambiguates convergence from rotation; detects hardening minorities
Live aggregates	Probes included	Self-correcting top set; probes stress-test the leaders
Tagging	samplingSlot + batchIndex on every evaluation	Keeps the causal (frozen-cohort) analysis reconstructable for free

The method turns the evaluation stream the platform already collects into a continuous, participant-invisible measurement of whether the deliberation is converging — answering, in near real time and at zero added burden, the question deliberative events exist to answer.